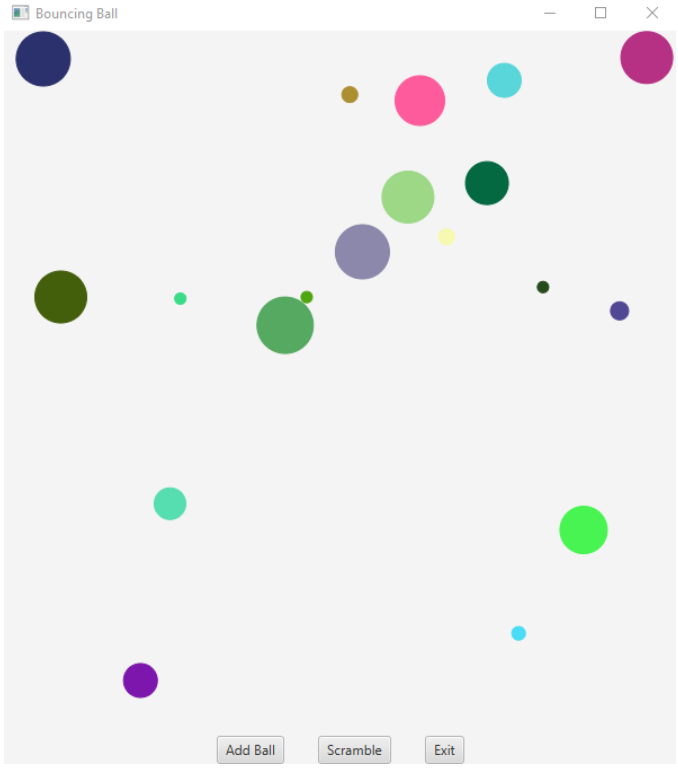
Multiple Bouncing Balls

In this lab, we will extend the single bouncing ball problem to multiple balls. The additional balls will have random size, random color, random starting direction and random speed. You will also provide controls to increase and decrease the number of bouncing balls dynamically and to alter the direction and speed of a selected ball.



Part I: (5pts):

1. Create a new JavaFX Application and copy the BouncePane and Ball classes from the single ball problem to the project directory.
2. Add a new class named MainPane that extends the BorderPane class. Add a property of type BouncePane to your MainPane class and add it to the center zone of the MainPane.
3. Modify the application start() method so that it instantiates a MainPane and displays it in the stage. **Show the single bouncing ball solution to your TA for your Part I points.**

Part II: (5pts)

1. Modify the BouncePane so that it contains a new ArrayList<Ball> property named *balls*.
2. Add a new method, *+addNewBall():void*, that creates a new Ball object, gives it a random rgb color between 0,0,0 and 255,255,255 a random radius between 5 and 30 and a random speed between 1 and 10. This method will also add the ball to the ArrayList and to the BouncePane. **Note:** It will prove helpful if you create a method that takes a ball object as a parameter and assigns random values for color, size, speed, and direction. This will be needed in several parts of the application.
3. Add a new method, *+moveBalls():void,* that moves each ball in the ArrayList. This is the method that will be called inside of your Timer handler to animate the bouncing balls.
4. Modify the collision code so that it checks each ball to see if it has collided with a wall. For this lab, we will not worry about ball-to-ball collisions (If you get time, go for it!!!).
5. Modify the MainPane constructor so that it creates 2 or 3 balls. They should all move and bounce within the BouncePane. **Show the multiple bouncing balls to your TA for Part II points.**

Part III: (5pts)

1. Add three new button properties to the MainPane with following text labels: Add Ball, Scramble, Exit. Place these buttons in a HBox and add the HBox to the bottom zone of the MainPane.

2. In the BouncePane class, add a method named +scramble():void. This method should change the color, size, direction, and size of the active balls.
6. Add anonymous listeners using lambda expressions that will cause the Add Ball button to call the addNewBall() method, the Scramble button to call the scramble() method, and the Exit button to Exit the application. **Show the functioning buttons to the TA for Part III points.**
3.

Part IV (5pts):

1. Add a private method in the BouncePane with three parameters (a Ball, x coordinate and y coordinate) and a boolean return type. The method should determine whether the (x, y) location is within the ball. You can just apply the distance formula between the center of the ball and the given point. If the distance is less than the ball radius, then the point is within the ball.
2. Add an inner class handler for mouse pressed (not mouse clicked) in your BouncePane class and register it with the BouncePane (i.e. this.setMousePressed(YourMouseHandlerObject)). The handler should check each ball to see if it was clicked by checking to see if the (x,y) location returned in the MouseEvent object is within any of the balls in the ArrayList. If a ball is clicked, then randomly change its color, size, speed and direction. **Show the functioning mouse pressed handler to a TA for your final points.**